Section 4: Optimistic Locking with @Version

1. What is a race condition?

A race condition happens when two operations read the same data simultaneously, make decisions based on it, and both try to write — each unaware the other exists.

The exact ridesharing scenario:

```
DB state: ride #42, status=OFFER_PENDING, offered to Driver 7

Thread A (Driver 7 accepts):        Thread B (offer expiry job):
reads ride #42 — status=PENDING     reads ride #42 — status=PENDING
sets status = MATCHED               sets status = OPEN (re-queue)
writes to DB                        writes to DB

Result: last write wins silently — corrupted data, no error thrown
```

2. Optimistic vs Pessimistic locking

	Optimistic	Pessimistic
Assumption	Conflicts are rare	Conflicts are frequent
Mechanism	Version column check on write	Lock row on read (SELECT FOR UPDATE)
On conflict	Throw OptimisticLockException	Other transaction waits
Performance	Fast — no locks held	Slower — rows locked during work
Best for	Ridesharing offer acceptance	Bank account debits

Why pessimistic locking is wrong for ridesharing

- Every driver reading any ride request would lock that row.
- Transaction holds the lock for ~200ms (validation, Redis calls, etc.).
- With 500 drivers online, 500 rows locked at any moment.
- Any driver looking at the same ride is stuck waiting — app grinds to a halt.
- Optimistic locking holds zero locks — conflict is detected only at write time.

3. How @Version works in JPA

```
@Entity
@Table(name = "ride_requests")
public class RideRequest {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private RideStatus status;
    private Long driverId;

    @Version
    private Long version;  // JPA manages this — never set it manually
}
```

What JPA does automatically:

- On load: reads current version from DB (e.g. version = 3)
- On UPDATE: adds version check to WHERE clause
- If 0 rows affected: throws OptimisticLockException
- If 1 row affected: increments version, commits

```
-- JPA generates this SQL automatically:
UPDATE ride_requests
SET status = 'MATCHED', driver_id = 7, version = 4
WHERE id = 42 AND version = 3   -- the version we read

-- If another transaction already updated it, version in DB is 4, not 3
-- WHERE matches 0 rows → OptimisticLockException thrown
```

4. The race condition — step by step with @Version

Step	Thread A (driver accepts)	Thread B (expiry job)
1. Both read	reads ride #42, version=2	reads ride #42, version=2
2. Both work	sets status=MATCHED	sets status=OPEN
3. A writes	UPDATE ... WHERE version=2 → 1 row, version becomes 3	still processing...
4. B writes	committed successfully	UPDATE ... WHERE version=2 → 0 rows affected
5. Outcome	ride is MATCHED ✓	OptimisticLockException thrown, rolled back

5. Service and controller code

```
@Transactional
public void acceptOffer(Long rideId, Long driverId) {
    RideRequest ride = rideRepo.findById(rideId).orElseThrow();

    if (ride.getStatus() != RideStatus.OFFER_PENDING) {
        throw new InvalidStateException("Offer is no longer valid");
    }

    ride.setStatus(RideStatus.MATCHED);
    ride.setDriverId(driverId);
    // @Version field incremented automatically at commit
```

```
    // If version conflict → OptimisticLockException before notification

    notificationService.notifyPassengerRideAccepted(
        ride.getPassengerId(), driverId, "5 minutes"
    );
}

@PostMapping("/api/offers/{rideId}/accept")
public ResponseEntity<?> acceptOffer(@PathVariable Long rideId,
                                     @RequestHeader("X-Driver-Id") Long driverId) {
    try {
        offerService.acceptOffer(rideId, driverId);
        return ResponseEntity.ok().build();

    } catch (OptimisticLockException |
             ObjectOptimisticLockingFailureException e) {
        // 409 Conflict — offer was already taken or expired
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body("Offer is no longer available");
    }
}
```

Why HTTP 409 Conflict?

- 400 Bad Request = client sent invalid data (not the case here).
- 404 Not Found = resource doesn't exist (it does — it's just taken).
- 409 Conflict = request is valid but conflicts with current resource state.
- 409 is semantically correct: the driver's request was valid, but the ride state changed.

Section 4 — what you learned

- Race condition: two threads read the same row, both write, last write silently wins.
- @Version adds a version column JPA manages automatically — never set it yourself.
- JPA adds AND version = N to every UPDATE — 0 rows affected means conflict detected.
- OptimisticLockException is thrown on conflict — transaction rolls back cleanly.
- Optimistic = no locks held, conflict detected at write time — right for ridesharing.
- Pessimistic = row locked on read — correct for banking, catastrophic for ridesharing at scale.
- Catch both OptimisticLockException and ObjectOptimisticLockingFailureException.
- Return HTTP 409 Conflict when the race is lost — not 400 or 404.

Next: Section 5 — Ride Request State Machine